

Everything new about JavaScript with ECMA Script 6 standard

One of the biggest changes to JavaScript was introduced in the ES6 standard and modern browsers have now completely adopted ES6. Here is everything that matters.

Xavier Das | feedback@digit.in

To most web developers out there, ES6 and JavaScript go hand in hand, after all, EcmaScript, is indeed the standard that JavaScript adheres to. And with the advent of EcmaScript 6, commonly referred to as ES6, which was the second major revision done to JavaScript in the year 2015, it revolutionised the age-old process of developing web applications, addressing some of the major drawbacks of the former JavaScript (ES1), and adding tonnes of features. The other reason for the overwhelming popularity of ES6 is that it is backwards compatible all the way to the initial release. Hence, older browsers, even if unable to render ES2015+ operations can be still be coded in the older JavaScript and would still run just as good in modern browsers. We will be looking at a few of the popular features in this article.

OPTIONAL CHAINING

This feature was introduced in the 2020 release of ES6 where JavaScript allows for us to access the value of a property in a chain of nested objects, simultaneously checking whether the property exists at all, and it ignores the check for the existence of every other object in the chain of reference until it encounters the optional chaining operator. The syntax of this operator is ?:

If the definition above did its best to confuse you, the example below shall do its best to clear it away.

```
const week = {
  mon: {
    open: 21,
    close: 22,
  },
  tues: {
    open: 21,
    close: 22,
  },
};

console.log(week.fri?.open);
console.log(week.fri.open);
```

Consider having the week object with mon and tues being the only two objects inside it and each of them having the properties open and close.

At this point, if somebody tries to refer to the open property of an object fri(which doesn't exist inside the week object), JavaScript immediately halts execution by throwing the error that says

Uncaught TypeError: Cannot read property 'open' of undefined.

On careful inspection we can figure out why this error comes in. We don't have any object called fri – it is undefined, and trying to open a property inside undefined is definitely an error.

As a way to work around this, we have the optional chaining operator placed right before open. What this does is to check and go deep into the nested call, in this case, there's only one level of nesting that it goes to, i.e., the week object and it checks for the existence of the object immediately to the left of the operator, which in this case is fri.

If fri exists, it proceeds forward into the chain and if it doesn't, the operator doesn't stop the flow of execution, rather it considers it as undefined and ignores it.

The statements that follow this line of code are then executed without any halt.